

1. ¿Qué es un IDE?

Definición · Componentes · Editor vs IDE

1.1 Definición

Un IDE (Integrated Development Environment, entorno integrado de desarrollo) reúne en una sola aplicación el editor de código, el compilador, el depurador y otras herramientas de apoyo al desarrollo.

¿Para qué sirve tenerlo todo integrado?

- Sin IDE: escribes en el editor → compilas en el terminal → depuras con otra herramienta → cambias constantemente de ventana.
- Con IDE: todo ocurre en la misma aplicación. El IDE detecta errores mientras escribes, antes de compilar.
 - Ejemplo: escribir `System.out.prtinln` en IntelliJ → el IDE lo subraya en rojo al instante. En el Bloc de notas no lo sabrías hasta ejecutar y ver el fallo.

La diferencia real no es de funciones: es de velocidad de trabajo. Un programador con un buen IDE detecta y corrige errores en segundos en lugar de minutos.

1.2 Componentes de un IDE

Componente	Función
Editor de código	Resaltado de sintaxis, autocompletado y detección de errores mientras escribes.
Compilador / Intérprete	Traduce el código fuente al formato que puede ejecutar la máquina o la JVM. En Java: javac.
Depurador	Pausa la ejecución en cualquier punto, muestra el valor de las variables y permite avanzar línea a línea.
Terminal integrado	Una consola dentro del propio IDE para ejecutar comandos sin salir de la aplicación.
Gestión de proyectos	Organiza los archivos del proyecto, gestiona dependencias externas y configura el proceso de construcción.
Control de versiones	Integración con Git: commits, historial de cambios y gestión de ramas sin salir del IDE.
Diseñador gráfico	En algunos IDEs, permite diseñar interfaces visuales arrastrando y soltando componentes.

1.3 Editor de código vs. IDE

No todo el mundo usa un IDE. Hay programadores que prefieren un editor ligero como VS Code o Vim. La diferencia es de grado, no de tipo:

Aspecto	Editor de código	IDE
Velocidad de arranque	Rápido (segundos)	Más lento (carga el proyecto)
Consumo de memoria	Bajo	Medio-alto
Funciones integradas	Básicas + extensiones a medida	Completas de serie
Curva de aprendizaje	Baja	Media
Cuándo conviene	Scripts, varios lenguajes, proyectos pequeños	Proyectos medianos/grandes con un lenguaje principal

Para aprender un lenguaje nuevo, un editor ligero reduce la fricción inicial. Para trabajar de forma profesional con un lenguaje durante meses, un IDE ahorra horas cada semana.

2. Tipos de entornos

Propietarios · Libres · Licencias educativas

2.1 IDEs propietarios

Un IDE propietario es desarrollado por una empresa que mantiene el control sobre el código fuente. El usuario no puede modificarlo ni redistribuirlo, y en entornos profesionales suele requerir pagar una licencia.

IDE	Empresa	Usos principales
IntelliJ IDEA Ultimate	JetBrains	Java, Kotlin, frameworks web empresariales
Visual Studio	Microsoft	C#, .NET, C++, desarrollo Windows
Xcode	Apple	Swift, Objective-C, apps iOS/macOS
WebStorm	JetBrains	JavaScript, TypeScript, Node.js

Suelen ofrecer la mejor integración con sus plataformas, soporte técnico oficial y funciones avanzadas. El coste puede ser un obstáculo, aunque muchos tienen licencias educativas gratuitas.

2.2 IDEs libres (código abierto)

Un IDE libre distribuye su código fuente bajo una licencia que permite estudiarlo, modificarlo y redistribuirlo. Generalmente es gratuito, aunque puede haber versiones de pago con funciones adicionales.

IDE	Licencia	Usos principales
Eclipse	EPL (Eclipse Public License)	Java, C/C++, PHP, muy extensible con plugins
NetBeans	Apache License 2.0	Java SE/EE, PHP, HTML5
IntelliJ IDEA Community	Apache License 2.0	Java, Kotlin (sin funciones empresariales)
VS Code / VSCodium	MIT (binarios: Microsoft)	Multiplataforma, requiere extensiones
PyCharm Community	Apache License 2.0	Python

JetBrains ofrece licencias gratuitas de todos sus IDEs profesionales (IntelliJ Ultimate, PyCharm, WebStorm...) para estudiantes y docentes mientras dure su vinculación con un centro educativo.

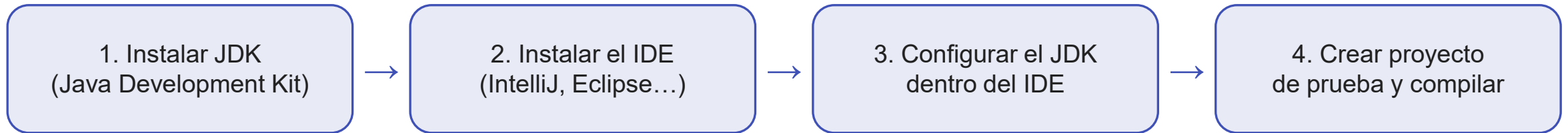
3. Instalación de un entorno

Requisitos previos · Proceso de instalación · Gestores de paquetes

3.1 Requisitos previos: el JDK

Antes de instalar el IDE, puede ser necesario instalar el runtime o SDK del lenguaje. Un runtime es el entorno que necesita el programa para ejecutarse. Un SDK (Software Development Kit) es el runtime más el compilador y el empaquetador.

Para Java: primero el JDK, luego el IDE



Sin el JDK instalado, el IDE no puede compilar ni ejecutar código Java.

Usa Eclipse Temurin (adoptium.net) en lugar de Oracle JDK. Es la distribución libre de OpenJDK más usada y recomendada. Para este módulo: Java 21 LTS.

3.2 Proceso de instalación genérico

Paso	Qué hacer
1. Descarga el instalador	Desde el sitio oficial del IDE. Elige la versión correcta para tu sistema operativo (Windows, macOS, Linux) y arquitectura (x86_64 o ARM).
2. Ejecuta el asistente	Sigue los pasos del instalador. Presta atención a la carpeta de instalación y a las opciones adicionales (asociar extensiones de archivo, añadir al PATH...).
3. Configura el SDK	En IntelliJ, al crear un proyecto nuevo te pide la ruta del JDK. Si no lo has instalado antes, el IDE puede descargarlo él mismo.
4. Crea un proyecto	Haz un "Hola mundo" y compílalo. Si funciona, la instalación es correcta.

La prueba final siempre es la misma: si puedes compilar y ejecutar un programa sencillo, el entorno está bien instalado.

3.3 Gestores de paquetes

En Linux y macOS es habitual instalar IDEs y runtimes a través de un gestor de paquetes: un programa que descarga, instala y actualiza software desde repositorios centralizados.

Sistema operativo	Gestor de paquetes	Ejemplo de uso
Ubuntu / Debian	apt	sudo apt install openjdk-21-jdk
Ubuntu (snap)	snap	sudo snap install intellij-idea-community --classic
macOS	Homebrew	brew install --cask intellij-idea-ce
Windows	Winget	winget install JetBrains.IntelliJIDEA.Community
Windows	Chocolatey	choco install intellijidea-community

En el instituto usaremos el instalador descargado directamente. Los gestores de paquetes son más habituales en entornos profesionales o en instalaciones masivas.

4. Módulos, plugins y extensiones

Qué son · Gestión en IntelliJ · Gestión en Eclipse

4.1 ¿Qué son los plugins?

La mayor parte de los IDEs no vienen con todo instalado de serie. Tienen un sistema de plugins (también llamados módulos, extensiones o complementos) que permite añadir funcionalidades adicionales cuando se necesitan.

¿Por qué no viene todo instalado?

Un IDE que incluyera soporte para todos los lenguajes, frameworks y herramientas existentes pesaría gigas y estaría lleno de funciones que nunca usarías. Los plugins permiten que el IDE sea ligero por defecto y que cada desarrollador lo adapte a su flujo de trabajo.

Qué puede añadir un plugin

- Soporte para un lenguaje que el IDE no incluye de serie (Python, Ruby, Rust...).
- Integración con un framework concreto (Spring, Django, Angular...).
- Herramientas de calidad de código: linters, formateadores automáticos.
- Temas visuales y fuentes de programación.
- Conectores con servicios externos: Jira, Docker, bases de datos...

4.2 Gestión de plugins en IntelliJ IDEA

Los plugins se gestionan desde File → Settings → Plugins (en macOS: IntelliJ IDEA → Settings).

Acción	Cómo hacerlo
Buscar e instalar	Pestaña Marketplace → busca el plugin → Install. Reinicia si lo pide.
Desactivar	Pestaña Installed → desactiva el toggle. El plugin queda instalado pero no consume recursos.
Desinstalar	Pestaña Installed → menú del plugin → Uninstall.
Actualizar	Pestaña Installed → Update disponible aparece destacado. Acepta la actualización.

No instales todo lo que encuentres. Cada plugin añadido consume memoria. Instala solo lo que vayas a usar activamente. Si algo lo pruebas puntualmente, desactívalo al terminar.

4.3 Gestión de plugins en Eclipse

En Eclipse, los plugins se instalan a través de `Help → Eclipse Marketplace` o `Help → Install New Software` (indicando la URL del repositorio del plugin). La filosofía es la misma: el IDE base es ligero y cada usuario añade lo que necesita.

Eclipse Marketplace

- Interfaz gráfica similar a una tienda de aplicaciones.
- Busca por nombre o categoría.
- Instala con un clic e indica qué dependencias se necesitan.
- Permite valorar y revisar plugins antes de instalar.
- Install New Software permite instalar desde una URL de repositorio directa.
- Útil para plugins no indexados en el Marketplace.
- Ejemplo: soporte para Kotlin en Eclipse se instala indicando la URL del repositorio de JetBrains.

Eclipse tiene una arquitectura muy modular: incluso las funciones más básicas son plugins que se pueden reemplazar o ampliar. Esto lo hace muy flexible pero también más complejo de configurar.

5. Personalización del entorno

Tema visual · Atajos · Estilo de código · Plantillas

5.1 Tema visual y fuente

Tema de color

- Los IDEs incluyen tema claro y oscuro de serie.
- Se pueden instalar temas adicionales como plugins (Dracula, One Dark, Nord...).
- Tema oscuro: reduce fatiga visual con poca luz ambiental.
- Tema claro: funciona mejor con mucha luz ambiental.
- Prueba ambos y quédate con el que te canse menos después de dos horas.
- Usa siempre una fuente monoespaciada: todos los caracteres tienen el mismo ancho, el código se alinea correctamente.
- Más usadas: JetBrains Mono, Fira Code, IBM Plex Mono.
- Fira Code tiene ligaduras: combina `->` o `!=` en un solo símbolo visual.
- Tamaño recomendado: 14-16 pt según pantalla.

El tema y la fuente no son caprichos estéticos: reducen la fatiga visual y facilitan reconocer la estructura del código a golpe de vista.

5.2 Atajos de teclado

Los atajos de teclado son la diferencia entre un programador fluido y uno que busca opciones en menús. Los IDEs permiten cambiar cualquier atajo y crear los propios.

Atajo (IntelliJ)	Acción
Ctrl + Space	Autocompletado básico
Ctrl + Shift + Space	Autocompletado inteligente (filtra por tipo esperado)
Ctrl + Alt + L	Reformatear el código según el estilo configurado
Shift + F10	Ejecutar el programa
Shift + F9	Depurar (iniciar el depurador)
Alt + Enter	Ver y aplicar correcciones rápidas sugeridas por el IDE
Ctrl + Click	Ir a la definición de una clase o método
Ctrl + Shift + F	Buscar un texto en todo el proyecto

No intentes memorizar 50 atajos a la vez. Elige los 5 que más tiempo te harían ahorrar y úsalos conscientemente durante una semana. Luego añade 5 más.

5.3 Estilo de codificación

El estilo de codificación define cómo se formatea el código automáticamente: espacios de indentación, dónde van las llaves, máximo de caracteres por línea... Los IDEs aplican estas reglas automáticamente al guardar o al reformatear.

¿Por qué importa en equipo?

- Si cada miembro del equipo usa un estilo diferente, los commits de Git mezclan cambios de lógica con cambios de formato.
- El historial se vuelve difícil de leer: ¿este commit cambia algo importante o solo mueve llaves?
- Con un estilo compartido configurado en el proyecto, el código tiene el mismo aspecto sin importar quién lo haya escrito.
 - Google Java Style y Oracle/Sun Style son las dos guías más extendidas en Java. IntelliJ permite importarlas como un archivo XML.

En IntelliJ: File → Settings → Editor → Code Style → Java → importar esquema XML del equipo.

5.4 Plantillas de código (Live Templates)

Las plantillas de código son fragmentos predefinidos que se expanden al escribir una abreviatura y pulsar Tab. Puedes crear las tuyas para cualquier bloque que repitas frecuentemente.

Ejemplos de plantillas de serie en IntelliJ

Abreviatura	Se expande en
sout	<code>System.out.println();</code>
soutv	<code>System.out.println("variable = " + variable);</code>
fori	<code>for (int i = 0; i < array.length; i++) { ... }</code>
iter	<code>for (Tipo elemento : coleccion) { ... }</code>
psvm	<code>public static void main(String[] args) { ... }</code>

Puedes crear tus propias plantillas en **Settings → Editor → Live Templates**. Útil para estructuras que escribes varias veces al día: bloques de logging, excepciones, tests, etc.

6. Actualización del entorno

Canales de actualización · Cómo actualizar en IntelliJ · Plugins

6.1 Canales de actualización

Los IDEs se actualizan con regularidad: nuevas funciones, mejoras de rendimiento, soporte para nuevas versiones de lenguajes y correcciones de errores. Gestionarlas bien evita sorpresas.

Canal	Estabilidad	Para quién
Stable / Release	Alta — pasa por todas las fases de prueba antes de publicarse	Uso diario, proyectos en producción. Siempre el canal de trabajo.
EAP (Early Access Program)	Media — versiones previas a la release con funciones nuevas	Usuarios que quieren explorar novedades antes que nadie.
Nightly	Baja — compilaciones automáticas del código en desarrollo	Contribuidores al IDE que necesitan probar cambios del día.

Para el trabajo habitual: usa siempre el canal Stable. Los canales EAP o Nightly son para explorar, no para trabajar. Una versión inestable en mitad de un proyecto importante puede costarte horas.

6.2 Actualización en IntelliJ IDEA

Actualización automática del IDE

- IntelliJ comprueba actualizaciones al arrancar y muestra una notificación cuando hay una nueva versión.
- Para ver y configurar el canal: File → Settings → Appearance & Behavior → System Settings → Updates.
- Acepta la actualización cuando aparezca. El IDE descarga e instala en segundo plano.

Actualización de plugins

- Los plugins se actualizan de forma independiente al IDE.
- Cuando hay actualizaciones pendientes, IntelliJ lo notifica en File → Settings → Plugins.
- Actualizar los plugins regularmente corrige errores y mantiene la compatibilidad con las versiones nuevas del IDE.

Antes de actualizar en mitad de un proyecto importante, comprueba que los plugins críticos que usas son compatibles con la nueva versión. En el JetBrains Marketplace puedes ver qué versiones soporta cada plugin.

7. Edición de programas

Autocompletado · Detección de errores · Refactorización

7.1 Autocompletado inteligente

El autocompletado sugiere completaciones basándose en el contexto: no solo palabras que empiezan por lo que has escrito, sino las que tienen sentido en ese punto según los tipos de variables y los métodos disponibles.

Atajo	Tipo de autocompletado	Qué filtra
Ctrl + Space	Básico	Todas las opciones disponibles que coinciden con lo escrito.
Ctrl + Shift + Space	Inteligente	Solo las opciones del tipo correcto para ese punto del código.
Ctrl + P	Parámetros	Muestra los parámetros que espera el método en el que estás.
Alt + Enter	Corrección rápida	Sugiere arreglos para el error o advertencia en esa línea.

El autocompletado reduce los errores tipográficos y ayuda a descubrir métodos de clases que no conocías. Acostúmbrate a usarlo siempre en lugar de escribir el nombre completo a mano.

7.2 Detección de errores en tiempo real

El IDE analiza el código mientras escribes y marca los problemas antes de compilar. Hay dos niveles:

Errores de compilación (rojo)

- El código es sintácticamente incorrecto.
- Falta un paréntesis, un tipo no coincide, un método no existe.
- El programa NO puede compilarse hasta corregirlos.

Advertencias (amarillo)

- El código es válido pero hay algo sospechoso.
- Variable no usada, posible NullPointerException, import innecesario...
- El programa compila pero puede fallar en ejecución.

Pulsa Alt + Enter sobre cualquier error o advertencia para ver las correcciones rápidas que propone el IDE. En muchos casos, aplica el arreglo automáticamente con un solo clic.

La barra lateral derecha del editor muestra un resumen del estado de todo el archivo: verde = sin problemas, amarillo = advertencias, rojo = errores.

7.3 Refactorización automática

Refactorizar es reestructurar el código para mejorar su calidad interna sin cambiar lo que hace externamente. Los IDEs hacen los cambios en todos los archivos afectados a la vez, evitando errores de copiar-pegar.

Refactorización	Qué hace	Atajo
Rename	Cambia el nombre de una variable, clase o método en todos los sitios donde se usa.	Shift + F6
Extract Method	Saca un bloque de código a un método nuevo con el nombre que indiques.	Ctrl + Alt + M
Extract Variable	Convierte una expresión en una variable con nombre.	Ctrl + Alt + V
Move	Mueve una clase a otro paquete actualizando todos los imports automáticamente.	F6
Inline	Lo contrario de Extract: sustituye el uso de una variable o método por su contenido directo.	Ctrl + Alt + N

Renombrar una clase a mano (buscar-reemplazar en todos los archivos) es propenso a errores. El IDE hace los cambios de forma segura porque entiende la estructura del código, no solo el texto.

8. Generación de ejecutables

Distintos lenguajes en el mismo IDE · Mismo código en distintos IDEs

8.1 Distintos lenguajes en el mismo IDE

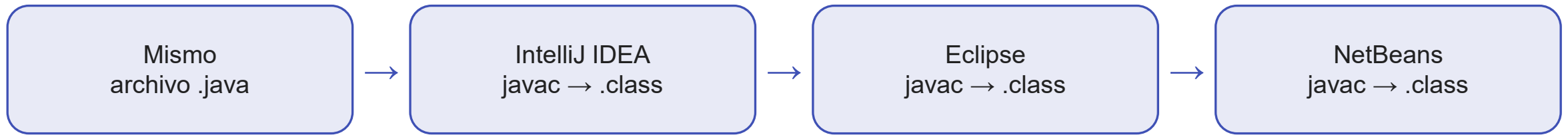
Un IDE como IntelliJ IDEA, con los plugins adecuados, puede compilar y ejecutar proyectos en varios lenguajes desde la misma aplicación. No hace falta tener un IDE diferente para cada lenguaje.

Lenguaje	En IntelliJ Community	Estado
Java	Incluido de serie	Sin instalar nada
Kotlin	Incluido de serie	Sin instalar nada
Python	Plugin: Python Community Edition (oficial de JetBrains)	Instalar desde Marketplace
JavaScript/TypeScript	Plugin: JavaScript and TypeScript	Instalar desde Marketplace
SQL	Plugin: Database Tools (completo en Ultimate)	Funcionalidad limitada en Community

Para cada lenguaje el proceso es el mismo: instalar el plugin, configurar el intérprete o SDK en el proyecto y después compilar/ejecutar normalmente.

8.2 El mismo código fuente en distintos IDEs

El mismo archivo .java se puede compilar y ejecutar desde Eclipse, NetBeans o IntelliJ sin modificarlo. Los IDEs no guardan el código en un formato propietario: son archivos de texto plano estándar.



El resultado es el mismo .class o .jar porque los tres IDEs llaman al mismo compilador de Java (javac) con los mismos archivos fuente.

Lo que Sí cambia entre IDEs

- Los archivos de proyecto internos: .idea/ en IntelliJ, .project y .classpath en Eclipse, nbproject/ en NetBeans.
- Esos archivos NO forman parte del código fuente y NO deben subirse al repositorio si el equipo usa IDEs distintos.
 - Añade al .gitignore: .idea/ .classpath .project nbproject/ *.iml

9. Herramientas y automatización

Maven · Gradle · Ant · Run Configurations

9.1 Herramientas de construcción (build tools)

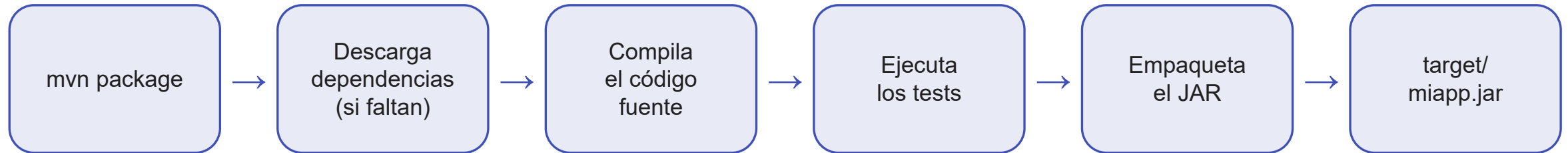
Compilar a mano un proyecto con decenas de archivos y librerías externas sería lento y propenso a errores. Las build tools automatizan todo el proceso.

Herramienta	Descripción	Archivo
Maven	Gestiona dependencias y define el proceso de compilación en XML. Tiene una estructura de carpetas fija que todos los proyectos Maven comparten (src/main/java, src/test/java...).	pom.xml
Gradle	Parecido a Maven pero más flexible. Usa un lenguaje de scripting (Groovy o Kotlin) en lugar de XML puro. Suele ser más rápido en proyectos grandes.	build.gradle
Ant	El más antiguo de los tres. Basado en XML pero sin gestión de dependencias propia. Ya casi no se usa en proyectos nuevos, pero puede aparecer en proyectos heredados.	build.xml

IntelliJ y NetBeans detectan automáticamente el pom.xml o build.gradle al abrir el proyecto y descargan las dependencias sin que tengas que hacer nada.

9.2 ¿Qué hace mvn package?

Con un solo comando (mvn package o gradle build), el proyecto se compila, se prueba y se empaqueta. Esto es especialmente útil para reproducir el proceso en otro ordenador o en un servidor.



Ciclo de vida de Maven (fases en orden)

Fase	Qué hace
compile	Compila el código fuente.
test	Ejecuta los tests unitarios (no detiene si fallan en modo exploratorio).
package	Empaqueta el compilado en .jar o .war.
install	Instala el paquete en el repositorio local (~/.m2).
deploy	Sube el paquete a un repositorio remoto compartido.

9.3 Configuraciones de ejecución

Los IDEs permiten definir Run Configurations: configuraciones guardadas que especifican qué clase ejecutar, con qué argumentos, qué variables de entorno usar... Puedes tener varias y cambiar entre ellas con un clic.

Casos de uso habituales

- Ejecutar en modo desarrollo: con la base de datos local y logs detallados.
- Ejecutar los tests: lanza solo la suite de tests, sin arrancar toda la aplicación.
- Ejecutar un servidor local: arranca un servidor embebido (Tomcat, Spring Boot...) apuntando a una carpeta concreta.
- Depurar: la misma configuración pero con el depurador activado, sin tener que cambiar nada más.

En IntelliJ: barra de herramientas superior → selector de configuración → Edit Configurations para crear o modificar.

10. Comparativa de entornos

Eclipse · NetBeans · IntelliJ IDEA Community · VSCodium

10.1 Tabla comparativa

Criterio	Eclipse	NetBeans	IntelliJ Community
Desarrollador	Fundación Eclipse	Apache	JetBrains
Licencia	EPL (libre)	Apache 2.0 (libre)	Apache 2.0 (libre)
Lenguajes	Java, C/C++, PHP + plugins	Java SE/EE, PHP, HTML5	Java, Kotlin
Gestión de plugins	Marketplace + Update Sites	Plugin Manager	JetBrains Marketplace
Build tools	Maven, Gradle, Ant (con plugins)	Maven, Gradle, Ant (integrados)	Maven, Gradle (integrados)
Facilidad de uso	Media — interfaz compleja	Alta — buena para aprender	Muy alta — interfaz moderna
Rendimiento	Medio	Medio	Alto
Mejor para	Proyectos empresariales grandes	Aprendizaje, Java EE	Productividad diaria

10.2 Características comunes y diferencias clave

Todos los IDEs tienen en común

- Editor con resaltado de sintaxis y autocompletado.
- Compilación y ejecución integradas.
- Depurador con breakpoints y seguimiento de variables.
- Integración con Git.
- Sistema de plugins o extensiones.
- Gestión de proyectos y dependencias.
- **Detección de errores:**
 - IntelliJ destaca con sugerencias más precisas e inteligentes.
- **Velocidad:**
 - Eclipse se ralentiza más en proyectos grandes. IntelliJ aguanta mejor.
- **Aspecto:**
 - IntelliJ tiene la interfaz más moderna. Eclipse y NetBeans muestran su antigüedad en algunas pantallas.
- **Para qué sirve cada uno:**

El mismo proyecto Java con Maven se puede abrir en los tres IDEs sin cambiar una sola línea de código. Lo único que cambia son los archivos de configuración internos del IDE (.idea/, .project...), que no forman parte del código fuente.